

基于RAG的半导体文档知识库系统

项目背景: 针对半导体领域大模型知识滞后、幻觉率高的问题（传统方案问答准确率仅62%），构建一个结合技术论文/研报的RAG系统，实现实时知识更新与可验证回答，支撑内部投研人员高效获取精准信息。

技术栈: LangChain、LangGraph、Milvus、UnstructuredLoader、PyMuPDF、DeepSeek-V3、RAGAS。

项目实施:

- 1、向量数据库在用弹性、分布式部署的Milvus数据库集群，采用Docker部署，可弹性增加节点。
- 2、使用PyMuPDF解析PDF/PPT文档。通过Linux服务器的私有化部署UnstructuredLoader。使用UnstructuredLoader加载非结构化文档（如 PDF、Word、HTML 等）。设置 hi_res策略来提取章节层级、表格等。通过Tesseract（OCR）提取图片信息。采用章节段落分块 + SemanticChunker语义分块相结合的方式来解决表格或者内容跨页断裂问题。
- 3、采用私有化部署的bge-large-zh-v1.5和BM25对Chunk进行密集向量化。创建密集向量Dense和稀疏向量Sparse的索引。其中稀疏向量索引采用DAAT算法，控制词频饱和度和参数BM25_k1，加速查询性能。稠密向量索引设置构建图时每个节点的最大邻接数M为30，efConstruction=64，提高搜索广度和精度。实现"向量+关键词"混合检索。
- 4、通过LangGraph定制 workflow，实现动态路由能力，根据上下文语音去不同的知识库或者网络进行检索。
- 5、引入Corrective RAG和Adaptive RAG构建两层评估体系。第一层：从知识库中检索出来的doc进行相关性评估。评估不达标通过Transform_Query触发重新检索。第二层：在输出答案之前进行输入的关联性评估。全部评估通过最终输出答案。

1 负责markdown文件的解析和切片的类

语义分割

相比传统的字符分割或token分割，语义分割的优势：

1. **保持语义完整性** - 不会在句子中间或重要概念中间分割
2. **基于内容而非长度** - 根据实际语义变化点分割
3. **更适合RAG系统** - 检索时能保持更好的语义一致性

定义 `MarkdownParser` 类专门用于解析和分割 Markdown 文件内容

1.1 初始化方法

- 使用 `SemanticChunker` 初始化一个文本分割器
- 参数说明：
 - `openai_embedding`: 使用的嵌入模型（从外部导入）
 - `breakpoint_threshold_type="percentile"`: 使用百分位数作为分割阈值
- 这个分割器将用于后续对长文档的语义分割

```
def __init__(self):
    self.text_splitter = SemanticChunker(
        openai_embedding, breakpoint_threshold_type="percentile"
    )
```

1.2主方法 parse_markdown_to_documents

主要的处理流程，分为三步，每一步都记录日志，方便调试和监控：

1. 解析Markdown文件 (`parse_markdown`)
2. 合并标题和内容 (`merge_title_content`)
3. 对长文档进行语义分割 (`text_chunker`)

```
def parse_markdown_to_documents(self, md_file: str, encoding='utf-8') -> List[Document]:
    documents = self.parse_markdown(md_file)
    log.info(f'文件解析后的docs长度: {len(documents)}')

    merged_documents = self.merge_title_content(documents)
    log.info(f'文件合并后的长度: {len(merged_documents)}')

    chunk_documents = self.text_chunker(merged_documents)
    log.info(f'语义切割后的长度: {len(chunk_documents)}')
    return chunk_documents
```

1.3Markdown解析方法 parse_markdown

- 使用 `UnstructuredMarkdownLoader` 加载Markdown文件
- 参数说明：
 - `mode='elements'`: 将Markdown按元素解析
 - `strategy='fast'`: 使用快速解析策略
- 使用 `lazy_load()` 方法惰性加载文档，然后转换为列表
- 返回一个 `Document` 对象列表

```
def parse_markdown(self, md_file: str) -> List[Document]:
    loader = UnstructuredMarkdownLoader(
        file_path=md_file,
        mode='elements',
        strategy='fast'
    )
    docs = []
    for doc in loader.lazy_load():
        docs.append(doc)
    return docs
```

1.4合并标题和内容方法 merge_title_content

主要功能是处理Markdown文档解析后的元素，将标题(Title)与其相关内容合并，构建文档的层次结构：

1. 移除文档元数据中的 'languages' 字段（根据需要）
2. 处理三种类型的文档元素：
 - 独立的叙述文本 (`NarrativeText` 且无 `parent_id`，即单独成段，无父元素)
 - 标题 (`Title`)
 - 标题下的内容 (有 `parent_id` 的非标题内容)
3. 构建标题的层级关系 (使用 `->` 连接)

4. 将内容合并到对应的标题下

5. 最终返回合并后的文档列表

```
def merge_title_content(self, datas: List[Document]) -> List[Document]:
    merged_data = [] #用于存储最终合并后的文档
    parent_dict = {} # 是一个字典，保存所有的父document， key为当前父document的ID,用于临时存储标题文档

    for document in datas:
        metadata = document.metadata
        if 'languages' in metadata:
            metadata.pop('languages')

        parent_id = metadata.get('parent_id', None) #获取父元素ID，表示当前元素属于哪个父元素
        category = metadata.get('category', None) #元素类型，如'Title'或'NarrativeText'
        element_id = metadata.get('element_id', None) #当前元素的唯一标识符

        # 处理内容document
        if category == 'NarrativeText' and parent_id is None:
            merged_data.append(document) #直接添加到结果列表merged_data中

        # 处理标题
        if category == 'Title':
            document.metadata['title'] = document.page_content #将标题内容保存到metadata
            # 的'title'字段
            if parent_id in parent_dict:
                document.page_content = parent_dict[parent_id].page_content + ' -> ' +
                document.page_content #如果这个标题有父标题，构建层级标题字符串(用" -> "连接)
                parent_dict[element_id] = document #将当前标题文档存入字典，key为当前元素的ID

            # 处理标题下的内容
            if category != 'Title' and parent_id: #如果不是标题但有父元素(即标题下的内容):
                parent_dict[parent_id].page_content = parent_dict[parent_id].page_content + ' '
                + document.page_content #将内容追加到父标题的page_content中
                parent_dict[parent_id].metadata['category'] = 'content' #将父元素的category标记
                为'content'

            # 将处理后的标题文档加入结果
            if parent_dict is not None:
                merged_data.extend(parent_dict.values()) #将所有处理过的标题文档(存储在parent_dict中)添加
                到结果列表

    return merged_data
```

1.5 文本分割方法 text_chunker

- 对文档列表中的每个文档进行检查
- 如果文档内容长度超过5000字符，使用 `SemanticChunker` 进行语义分割
- 否则直接保留原文档
- 返回处理后的文档列表

```
def text_chunker(self, datas: List[Document]) -> List[Document]:
    new_docs = []
    for d in datas:
        if len(d.page_content) > 5000: # 内容超出了阈值，则按照语义再切割
            new_docs.extend(self.text_splitter.split_documents([d]))
            continue
        new_docs.append(d)
    return new_docs
```

2 负责将文档数据存储在 Milvus 向量数据库中的类

这个类负责将文档数据存储在 Milvus 向量数据库中，支持稠密向量(dense)和稀疏向量(sparse)两种索引方式。

2.1 类结构和初始化

- 类的主要功能是将文档数据插入到 Milvus 数据库
- `vector_store_saved` 属性用于保存 Milvus 连接实例

```
class MilvusVectorSave:
    """把新的document数据插入到数据库中"""

    def __init__(self) -> object:
        """自定义collection的索引"""
        self.vector_store_saved: Milvus = None
```

2.2 创建集合(Collection)方法

`create_collection()` 方法负责创建 Milvus 集合(相当于关系型数据库的表)并定义其结构:

2.2.1 创建客户端和模式(Schema)

- 创建 Milvus 客户端连接
- 创建新的模式(Schema)对象

```
client = MilvusClient(uri=MILVUS_URI)
schema = client.create_schema()
```

2.2.2 定义字段结构

字段说明:

- `id`: 主键，自动生成
- `text`: 文本内容，使用 jieba 分词器进行分词
- `category`: 文档类别
- `source`: 数据来源
- `filename`: 文件名
- `filetype`: 文件类型
- `title`: 文档标题
- `category_depth`: 类别深度
- `sparse`: 稀疏向量(用于BM25检索)
- `dense`: 稠密向量(512维，用于向量检索)

```

schema.add_field(field_name='id', datatype=DataType.INT64, is_primary=True, auto_id=True)
schema.add_field(field_name='text', datatype=DataType.VARCHAR, max_length=6000,
                  enable_analyzer=True, analyzer_params={"tokenizer": "jieba", "filter":
["cnalphanumonly"]})
schema.add_field(field_name='category', datatype=DataType.VARCHAR, max_length=1000)
schema.add_field(field_name='source', datatype=DataType.VARCHAR, max_length=1000)
schema.add_field(field_name='filename', datatype=DataType.VARCHAR, max_length=1000)
schema.add_field(field_name='filetype', datatype=DataType.VARCHAR, max_length=1000)
schema.add_field(field_name='title', datatype=DataType.VARCHAR, max_length=1000)
schema.add_field(field_name='category_depth', datatype=DataType.INT64)
schema.add_field(field_name='sparse', datatype=DataType.SPARSE_FLOAT_VECTOR)
schema.add_field(field_name='dense', datatype=DataType.FLOAT_VECTOR, dim=512)

```

2.2.3添加BM25函数

- 定义BM25函数，用于生成稀疏向量
- 输入是 text 字段，输出到 sparse 字段

```

bm25_function = Function(
    name="text_bm25_emb", # 函数名
    input_field_names=["text"], # 输入字段
    output_field_names=["sparse"], # 输出字段
    function_type=FunctionType.BM25 # 使用BM25算法
)
schema.add_function(bm25_function)

```

2.2.4定义索引参数

索引说明：

- **稀疏索引：**
 - 使用倒排索引(SPARSE_INVERTED_INDEX)
 - 使用BM25评分算法
 - 参数调优: k1=1.2, b=0.75
- **稠密索引：**
 - 使用HNSW图索引
 - 使用内积(IP)作为相似度度量
 - 参数: M=16(邻接节点数), efConstruction=64(构建时的搜索范围)

```

index_params = client.prepare_index_params()

# 稀疏向量索引
index_params.add_index(
    field_name="sparse",
    index_name="sparse_inverted_index",
    index_type="SPARSE_INVERTED_INDEX",
    metric_type="BM25",
    params={
        "inverted_index_algo": "DAAT_MAXSCORE",
        "bm25_k1": 1.2,
        "bm25_b": 0.75
    }
)

# 稠密向量索引
index_params.add_index(
    field_name="dense",

```

```

        index_name="dense_inverted_index",
        index_type=IndexType.HNSW,
        metric_type=MetricType.IP,
        params={"M": 16, "efConstruction": 64} # M :邻接节点数, efConstruction: 搜索范围
    )

```

2.2.5清理并创建集合

- 如果集合已存在，先释放、删除索引和集合
- 然后创建新的集合

```

if COLLECTION_NAME in client.list_collections():
    # 先释放，再删除索引，再删除collection
    client.release_collection(collection_name=COLLECTION_NAME)
    client.drop_index(collection_name=COLLECTION_NAME, index_name='sparse_inverted_index')
    client.drop_index(collection_name=COLLECTION_NAME, index_name='dense_inverted_index')
    client.drop_collection(collection_name=COLLECTION_NAME)

client.create_collection(
    collection_name=COLLECTION_NAME,
    schema=schema,
    index_params=index_params
)

```

2.3创建连接方法

- 创建 LangChain 与 Milvus 的连接
- 参数说明:
 - `embedding_function=bge_embedding`: 使用BGE嵌入模型生成稠密向量
 - `builtin_function=BM25BuiltInFunction()`: 使用内置BM25函数生成稀疏向量
 - `vector_field=['dense', 'sparse']`: 指定向量字段
 - `consistency_level="Strong"`: 强一致性
 - `auto_id=True`: 自动生成ID
 - `connection_args={"uri": MILVUS_URI}`: 连接参数

```

def create_connection(self):
    """创建一个Connection: milvus + langchain"""
    self.vector_store_saved = Milvus(
        embedding_function=bge_embedding,
        collection_name=COLLECTION_NAME,
        builtin_function=BM25BuiltInFunction(),
        vector_field=['dense', 'sparse'],
        consistency_level="Strong",
        auto_id=True,
        connection_args={"uri": MILVUS_URI}
    )

```

2.4添加文档方法

- 将文档列表添加到 Milvus 集合中
- 会自动调用嵌入模型生成向量

```

def add_documents(self, datas: List[Document]):
    """把新的document保存到Milvus中"""
    self.vector_store_saved.add_documents(datas)

```

3 采用分布式，多进程的方式把海量数据写入Milvus数据库

定义了两个函数，用于实现一个分布式多进程系统，用于高效处理大量Markdown文档并将其存储到Milvus向量数据库中。

采用生产者-消费者模式，由两个主要进程组成：

1. **文件解析进程** (`file_parser_process`):
 - 扫描指定目录下的Markdown文件
 - 使用 `MarkdownParser` 解析文件内容
 - 将解析结果分批放入队列
2. **数据库写入进程** (`milvus_writer_process`):
 - 从队列获取文档批次
 - 使用 `MilvusVectorSave` 将文档写入Milvus
 - 维护写入计数和状态

3.1 文件解析进程函数

生产者进程函数，这个函数用于扫描目录下的Markdown文件，解析内容，分批发送到输出队列

输入参数:

- `dir_path`: 要扫描的目录路径
- `output_queue`: 多进程队列，用于输出解析结果
- `batch_size`: 批处理大小，默认20个文档一批

1. 文件扫描阶段

- 使用列表推导式高效扫描目录
- `os.path.join` 确保跨平台路径兼容性
- 只处理 `.md` 后缀的Markdown文件

```
# 获取目录下所有.md文件
md_files = [
    os.path.join(dir_path, f)
    for f in os.listdir(dir_path)
    if f.endswith('.md')
]
```

- 空目录检查
- 发送 `None` 作为进程终止信号
- 提前返回避免不必要的初始化

```
if not md_files:
    log.warning("警告: 未找到任何.md文件")
    output_queue.put(None) # 发送终止信号
    return
```

2. 初始化准备

- 创建Markdown解析器实例

- 初始化文档批处理列表

```
parser = MarkdownParser()
doc_batch = []
```

3. 文件循环解析

- 遍历每个Markdown文件
- `try-except` 块捕获单个文件解析错误
- 使用解析器的 `parse_markdown_to_documents` 方法
- 过滤掉空结果(`if docs:`)
- 当累积文档数达到 `batch_size` 时 (分批处理避免内存爆炸) :
 - `copy()` 防止引用问题
 - 放入输出队列
 - 清空当前批次

```
for file_path in md_files:
    try:
        docs = parser.parse_markdown_to_documents(file_path)
        if docs:
            doc_batch.extend(docs)

        # 达到批次大小时发送 到队列中
        if len(doc_batch) >= batch_size:
            output_queue.put(doc_batch.copy())
            doc_batch.clear() # 清空当前缓冲区的所有批次数据
    except Exception as e:
        log.error(f"解析失败 {file_path}: {str(e)}")
        log.exception(e)
```

4. 收尾处理

- 发送最后不满一批的剩余文档
- 发送终止信号
- 记录处理完成日志

```
# 发送剩余文档
if doc_batch:
    output_queue.put(doc_batch)

# 发送终止信号
output_queue.put(None)
log.info(f"解析完成, 共处理{len(md_files)}个文件")
```

3.2 从队列读取并写入Milvus进程函数

消费者进程函数, 负责从共享队列中获取文档数据并写入Milvus向量数据库。

参数: `input_queue`: 多进程安全队列, 用于接收待处理文档

1. 初始化阶段

数据库连接:

- 创建 `MilvusVectorSave` 实例
- 建立到Milvus的连接(`create_connection()`)

计数器初始化: `total_count` 用于统计写入总量

```
mv = MilvusVectorSave()
mv.create_connection()
total_count = 0
```

2. 主循环逻辑

无限循环：持续处理直到收到终止信号

队列消费：

- `input_queue.get()` 是阻塞调用，队列空时自动等待
- 收到 `None` 时跳出循环（生产者发送的终止信号）

类型安全校验： `isinstance(datas, list)` 确保数据格式正确

写入操作：

- 调用 `add_documents()` 批量写入
- 自动完成向量化并存入Milvus

计数与日志：实时更新写入总数

```
while True:
    try:
        datas = input_queue.get() # 阻塞的函数
        if datas is None: # 收到了终止的信号
            break

        if isinstance(datas, list):
            mv.add_documents(datas)
            total_count += len(datas)
            log.info(f"累计已写入: {total_count} 个文档")
    except Exception as e:
        log.error(f"写入数据是吧 ! ")
        log.exception(e)

log.info(f"写入进程结束，总计写入 {total_count} 个文档")
```

3.3主控制程序

多进程文档处理系统的主控制程序，负责协调文件解析和数据库写入两个子进程的工作。

1. 参数配置

- **md_dir:** 指定包含Markdown文件的目录路径
 - 使用原始字符串(r前缀)避免转义字符问题
 - 应确保该路径存在且包含.md文件
- **queue_maxsize:** 设置进程间队列的最大容量
 - 防止生产者过快导致内存溢出
 - 值20是一个经验值，平衡内存使用和吞吐量

```
if __name__ == '__main__':
    # 配置参数
    md_dir = r'E:\my_project\langchain_demo01\md' # Markdown文件目录
    queue_maxsize = 20 # 队列最大容量（防止内存溢出）
```

2. Milvus集合初始化

- 创建MilvusVectorSave实例
- 初始化Milvus集合(相当于数据库表)

- 包括定义schema、索引等
- 如果集合已存在会先删除重建

```
mv = MilvusVectorSave()
mv.create_collection()
```

3. 进程间通信队列

- 创建多进程安全队列
- 特性:
 - 线程/进程安全
 - 当队列满时, put操作会自动阻塞
 - 当队列空时, get操作会自动阻塞
- 最大容量限制为20, 防止内存过度使用

```
docs_queue = Queue(maxsize=queue_maxsize)
```

4. 创建并启动子进程

- **解析进程(parser_proc):**
 - 执行file_parser_process函数
 - 传入参数: md_dir和docs_queue
- **写入进程(writer_proc):**
 - 执行milvus_writer_process函数
 - 传入参数: docs_queue
- 启动顺序:
 1. 先启动解析进程(生产者)
 2. 再启动写入进程(消费者)

```
# 启动子进程
parser_proc = multiprocessing.Process(
    target=file_parser_process,
    args=(md_dir, docs_queue)
)
writer_proc = multiprocessing.Process(
    target=milvus_writer_process,
    args=(docs_queue,)
)

parser_proc.start()
writer_proc.start()
```

5. 进程同步与等待

- `join()` 方法阻塞主进程, 直到对应子进程结束
- 等待顺序:
 1. 先等待解析进程完成(所有文件处理完毕)
 2. 再等待写入进程完成(队列中的剩余数据处理完)
- 这样确保所有文档都被正确处理

```
parser_proc.join()
writer_proc.join()
```

4 定义检索器retriever_tool

创建了一个基于Milvus向量数据库的检索工具，用于在RAG(检索增强生成)系统中获取相关知识文档。

1. 初始化操作

- **MilvusVectorSave**: 自定义的Milvus向量存储类
- 创建Milvus连接实例并建立数据库连接

```
mv = MilvusVectorSave()
mv.create_connection()
```

2. 检索器(Retriever)配置

检索参数详解：

参数	类型	说明	推荐值
search_type	str	搜索类型	'similarity'(相似度)/'mmr'(最大边际相关性)
k	int	返回文档数量	根据需求调整(通常3-5)
score_threshold	float	相似度阈值	0.1-0.3(需实验确定)
ranker_type	str	排序算法	'rrf'(推荐)/'score'
ranker_params	dict	排序参数	RRF通常设k=60-100
filter	dict	元数据过滤	如{"category": "content"}

```
retriever = mv.vector_store_saved.as_retriever(
    search_type='similarity', # 仅返回相似度超过阈值的文档
    search_kwargs={
        "k": 4,
        "score_threshold": 0.1,
        "ranker_type": "rrf",
        "ranker_params": {"k": 100},
        'filter': {"category": "content"}
    }
)
```

3. 创建检索工具

参数说明：

1. **retriever**: 上一步配置的检索器实例
2. **name**: 工具名称(需唯一)，Agent将通过此名称调用工具
3. **description**: 工具的自然语言描述，关键作用包括：
 - 帮助LLM理解工具用途
 - 影响Agent是否/何时调用该工具
 - 应清晰说明工具覆盖的知识范围

```
retriever_tool = create_retriever_tool(
    retriever,
    'rag_retriever',
    '搜索并返回关于 ‘半导体和芯片’ 的信息，内容涵盖：半导体和芯片的封装、测试、光刻胶等'
)
```

5 方式一：基于RAG的智能助手

定义了一个基于检索增强生成(RAG)的智能助手系统，能够利用知识库工具回答问题。

使用的langchain框架，仅作演示。后面的全用langgraph框架

1. 创建提示词模板

- **系统消息**：定义助手角色和行为准则
- **聊天历史**：预留位置动态插入历史对话
- **用户输入**：接收用户问题
- **思考过程**：预留Agent的思考过程记录位置

```
prompt = ChatPromptTemplate.from_messages([
    ('system', '你是一个智能助手，尽可能的调用工具回答用户的问题'),
    MessagesPlaceholder(variable_name='chat_history', optional=True),
    ('human', '{input}'),
    MessagesPlaceholder(variable_name='agent_scratchpad', optional=True),
])
```

2. 定义Agent的创建与执行(langchain的框架)

- **create_tool_calling_agent**：创建能调用工具的Agent
 - 参数：LLM实例、工具列表、提示模板
- **AgentExecutor**：实际执行Agent逻辑的组件
 - 处理工具调用、响应生成等流程

```
agent = create_tool_calling_agent(llm, [retriever_tool], prompt)

executor = AgentExecutor(agent=agent, tools=[retriever_tool])
```

3. 对话历史管理

- **store字典**：内存存储各会话的历史记录
- **get_session_history**：获取或创建指定会话的历史记录
 - 每个session_id对应独立的聊天历史
 - 实现简单的会话隔离

```
store = {}

def get_session_history(session_id: str) -> BaseChatMessageHistory:
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]
```

4. 带历史记录的Agent

RunnableWithMessageHistory：为Agent添加历史记录功能

- 封装基础的executor
- 指定历史记录获取函数
- 定义输入和历史记录的key

```
agent_with_history = RunnableWithMessageHistory(
    executor,
    get_session_history,
    input_messages_key='input',
    history_messages_key='chat_history'
)
```

5. 调用案例

- **invoke调用**：执行带历史记录的Agent
- **输入格式**：
 - `input`：用户问题
 - `config`：配置会话ID('zs123')

```
resp2 = agent_with_history.invoke(
    {'input': '什么是EUV光刻机?'},
    config={'configurable': {"session_id": 'zs123'}}
)
print(resp2)
```

6 方式二：自我纠正的RAG—Corrective RAG

Corrective RAG 是 RAG（检索增强生成）的改进范式，通过引入 实时反馈与纠正机制，在生成过程中动态检测和修正错误，显著提升输出的准确性和可靠性。其核心思想是：

***“生成-评估-纠正”闭环：**在传统RAG的单次检索生成基础上，增加对生成内容的实时验证与迭代优化。

多粒度纠错：从事实准确性、逻辑一致性、上下文连贯性等维度进行自我修正。

此项目执行流程：首先进入Agent节点，由LLM判断是否需要调用检索器（工具节点）；调用检索器后，由路由函数判断内容是否合格；不合格会进入Rewrite节点由LLM根据检索出来的内容重写问题，然后返回Agent节点；若合格则会进入Generate节点生成最终内容

6.1定义状态

```
class AgentState(TypedDict):
    # add_messages 函数定义了应如何处理状态更新
    # add_messages 表示"追加"（append）
    messages: Annotated[list[BaseMessage], add_messages]
```

6.2定义节点

根据项目执行流程，需要定义的节点有四个：agent节点、retrieve检索器工具节点、rewrite重写问题节点、generate生成最终回答节点。

6.2.1agent节点

- 从state中获取messages，获取的是整个消息历史列表（包含所有之前的对话轮次）
- 给LLM绑定检索器工具
- 调用模型执行messages中最后一条消息，得到响应
- 将响应添加到messages中返回

```
# agent节点
def agent_node(state: AgentState):
    """
```

调用智能体模型基于当前状态生成响应。根据问题，它会决定使用检索工具检索，或者直接结束。

参数：

state (messages): 当前状态

返回：

dict: 更新后的状态，包含附加到消息中的智能体响应

"""

log.info("---开始进入工作流---")

messages = state["messages"]

model = llm.bind_tools([retriever_tool])

response = model.invoke([messages[-1]])

返回列表，因为这会添加到现有列表中

return {"messages": [response]}

[messages[-1]]：获取的是只包含最新一条消息的列表（最后一条用户输入）

6.2.2rewrite重写问题节点

- 从state中获取messages，获取的是整个消息历史列表（包含所有之前的对话轮次）
- 获取最后一条HumanMessage（反向遍历列表快速获得）
- 定义提示词（告诉大模型任务，传递给llm最新的问题，让其生成改进后的问题）
- 调用llm（一次基础调用，非对话模式，仅传入提示词），仅对用户最新问题进行改造（不结合检索到的知识）
- 将llm重新生成的问题传入messages并返回

```
def rewrite(state):
```

```
    """
```

```
    转换查询以生成更好的问题。
```

```
    参数：
```

```
        state (messages): 当前状态
```

```
    返回：
```

```
        dict: 包含重述问题的更新后状态
```

```
    """
```

```
    log.info("---转换查询---")
```

```
    messages = state["messages"]
```

```
    question = get_last_human_message(messages).content
```

```
    msg = [
```

```
        HumanMessage(
```

```
            content=f""" \n
```

```
分析输入并尝试理解潜在的语义意图/含义。 \n
```

```
这是初始问题：
```

```
\n ----- \n
```

```
{question}
```

```
\n ----- \n
```

```
请提出一个改进后的问题： """,
```

```
    )
```

```
]
```

```
    # 评分模型
```

```
    response = llm.invoke(msg)
```

```
    return {"messages": [response]}
```

```
def get_last_human_message(messages: List[BaseMessage]) -> HumanMessage:
```

```
    # 反向遍历列表，找到最后一个HumanMessage实例
```

```
    for message in reversed(messages):
```

```

    if isinstance(message, HumanMessage):
        return message
    raise ValueError("No HumanMessage found in the messages list")

```

6.2.3 generate生成最终回答节点

- 获取最后的问题（HumanMessage中）
- 获取messages中最后消息的内容（检索器检索到的内容）
- 定义提示词模板，让llm根据用户最新的问题与检索到的内容进行回答
- 将提示词、llm、将输出转化为纯字符串函数绑定为链
- 调用链，并传入问题与上下文
- 将大模型调用输出的字符串封装为AIMessage，并传入messages中返回

```

def generate(state):
    """
    生成答案

    参数:
        state (messages): 当前状态

    返回:
        dict: 包含重述问题的更新后状态
    """
    log.info("---生成最终的答案---")
    messages = state["messages"]
    question = get_last_human_message(messages).content
    last_message = messages[-1]

    docs = last_message.content

    # 提示模板
    prompt = PromptTemplate(
        template="你是一个问答任务助手。请根据以下检索到的上下文内容回答问题。如果不知道答案，请直接说明。回答保持简洁。\\n问题: {question} \\n上下文: {context} \\n回答: ",
        input_variables=["question", "context"],
    )
    # 处理链
    rag_chain = prompt | llm | StrOutputParser()
    # 执行
    response = rag_chain.invoke({"context": docs, "question": question})
    ai_message = AIMessage(content=response)
    return {"messages": [ai_message]}

```

6.3 定义图

所有需要的节点都已经定义好了。

还需要定义边，固定边就不说了，需要定义两个条件边：

1. 第一条是从agent节点出发，需要判断是否需要调用检索器工具，不需要则直接END，这条条件边的路由函数可以直接调用langgraph提供的路由函数。
2. 第二条条件边是在条用检索器工具后，需要判断检索出来的内容和用户提问相关性如何（调用llm来判断相关性），返回yes或着no，yes即为相关性好，则路由到generate节点生成最终答案。若为no，则路由到rewrite节点，由llm对用户问题进行改造，然后由固定边到agent节点重走garph。

6.3.1 定义调用检索器工具后的路由函数

- 定义数据模型类Grade，可以限制llm的回答必须为数据模型类中定义好的内容，即只能返回yes或者no
- 调用 `llm.with_structured_output` (langchain提供) 将LLM输出约束为特定结构Grade
- 提示词模板，告诉大模型，他的任务是判断用户问题与检索器检索出内容的相关性，需要传入这两个信息
- 将提示词与带结构化输出的llm绑定为链
- 获取用户最新问题，与检索器检索出的内容
- 调用链，并传入所需要的两个参数，获取llm的回答 (llm生成: {"binary_score": "yes"}，系统自动转换为: Grade(binary_score="yes"))
- 访问属性获取值 (yes或者no)
- 若为yes则返回generate，若为no则返回rewrite

```
# 数据模型
class Grade(BaseModel):
    """相关性检查的二元评分"""

    binary_score: str = Field(description="相关性评分 'yes' 或 'no'")

def grade_documents(state) -> Literal["generate", "rewrite"]:
    """
    判断检索到的文档是否与问题相关。
    参数:
        state (messages): 当前状态
    返回:
        str: 判断结果，文档是否相关
    """
    log.info("---检查document的相关性---")
    # 带结构化输出的LLM
    llm_with_structured = llm.with_structured_output(Grade)

    # 提示模板
    prompt = PromptTemplate(
        template="""你是一个评估检索文档与用户问题相关性的评分器。\\n
        这是检索到的文档: \\n\\n {context} \\n\\n
        这是用户的问题: {question} \\n
        如果文档包含与用户问题相关的关键词或语义含义，则评为相关。\\n
        给出二元评分 'yes' 或 'no' 来表示文档是否与问题相关。""",
        input_variables=["context", "question"],
    )

    # 处理链
    chain = prompt | llm_with_structured

    messages = state["messages"]
    last_message = messages[-1]

    question = get_last_human_message(messages).content
    docs = last_message.content

    scored_result = chain.invoke({"question": question, "context": docs})

    score = scored_result.binary_score

    if score == "yes":
        print("---输出: 文档相关---")
        return "generate"

    else:
```

```

print("---输出: 文档不相关---")
print(score)
return "rewrite"

```

6.3.2绘制图

- 定义一个新的 workflow
- 将四个节点添加进去
- 添加固定边: START->agent、rewrite->agent、generate->END
- 添加条件边: agent->retrieve/END (使用的是langgraph提供的默认的路由函数tools_condition)、

retrieve->rewrite/generate (使用上述定义好的路由函数)

- 添加记忆功能 (配置检查点为memory), 使用MemorySaver()内存存储, 临时存储 (进程结束时丢失)
- 设置会话配置config, 用于配置会话id
- 编译图, 执行图

```

# 定义一个新的 workflow
workflow = StateGraph(AgentState)

# 添加节点
workflow.add_node('agent', agent_node)
workflow.add_node('retrieve', ToolNode([retriever_tool]))
workflow.add_node('rewrite', rewrite)
workflow.add_node('generate', generate)

workflow.add_edge(START, 'agent')
workflow.add_conditional_edges(
    'agent',
    tools_condition,
    {
        'tools': 'retrieve',
        END: END
    }
)

workflow.add_conditional_edges(
    'retrieve',
    grade_documents,
)

workflow.add_edge('rewrite', 'agent')
workflow.add_edge('generate', END)

# 检查点让状态图可以持久化其状态
# 这是整个状态图的完整内存
memory = MemorySaver()

# 编译状态图, 配置检查点为memory, 配置中断点
graph = workflow.compile(checkpointer=memory)

# draw_graph(graph, 'graph_rag1-2.png')
config = {
    "configurable": {
        # 检查点由session_id访问
        "thread_id": str(uuid.uuid4()),
    }
}

```

```

_printed = set() # set集合，避免重复打印

# 执行工作流
while True:
    question = input('用户: ')
    if question.lower() in ['q', 'exit', 'quit']:
        log.info('对话结束，拜拜! ')
        break
    else:
        inputs = {
            "messages": [
                ("user", question),
            ]
        }
        events = graph.stream(inputs, config=config, stream_mode='values')
        # 打印消息
        for event in events:
            _print_event(event, _printed)

```

7 方式三：自适应RAG—Adaptive RAG

Adaptive RAG 是一种动态调整检索和生成策略的 RAG 增强范式，旨在根据输入查询的复杂性、上下文需求和数据分布，智能优化检索范围、生成方式和资源分配。其核心思想是：

动态适应性：不再采用固定的检索-生成流程，而是根据任务需求调整策略。

资源效率：在简单查询时减少计算开销，在复杂查询时增强检索和推理能力。

上下文感知：结合用户历史交互、领域知识等优化检索和生成。

项目的执行流程：首先就通过路由函数由llm判断用户问题需要联网搜索还是使用检索器检索。

1. 通过网络搜索：进入web_search节点，搜索出相关信息后传入generate节点，由generate节点生成答案后需要进入下一个路由函数，判断所生成的答案是否准确回答了用户的问题，返回yes或者no
 - yes：所生成的答案准确回答了用户的问题，则进入END节点，返回最终回答
 - no：所生成的答案不能准确回答用户的问题，则进入transform_query节点，由llm改造用户的问题，然后进入retrieve检索器节点，后续操作见2
 - 所生成的答案没有基于web_search节点搜索出的相关信息（存在幻觉的可能），则返回generate节点重新生成答案
2. 使用检索器检索：进入retrieve节点，通过检索器工具节点检索出内容后进入grade_document节点判断检索出的document和问题的相关性，得到过滤后的documents。然后进入路由函数：
 - 如果过滤后的documents为空，则路由到transform_query节点重新生成问题，并且将transform_count（生成次数+1）
 - 如果transform_count重新生成的次数大于或等于2，则进入web_search节点去网络中搜索
 - 过滤后的documents不为空，则进入generate节点生成最终回答

7.1 定义状态

不用messages，改为存储四个字段：

- question：直接定义一个question字段，存储当前处理的用户问题，这样就不用每次都调用函数从messages中提出用户的最新问题了。
- generation：定义generation字段存储模型的回答文本
- transform_count：用于记录重新生成问题的次数，可以限定超过次数就不再重新生成
- documents：列表，存储从检索器中检索到的document

```

class GraphState(TypedDict):

```

```

"""
表示图处理流程的状态信息

属性说明:
    question: 用户提出的问题文本
    generation: 语言模型生成的回答文本
    transform_count: 转换查询的次数
    documents: 检索到的相关文档列表
"""

question: str # 存储当前处理的用户问题
transform_count: int # 转换查询的次数
generation: str # 存储LLM生成的回答内容
documents: List[Document] # 存储检索到的文档内容列表

```

7.2定义节点

由项目执行流程可知，一共需要定义以下节点：web_search网络搜索节点、generate最终答案生成节点、transform_query重新生成问题节点、retrieve检索器工具节点、garde_document检索出的文档评价节点

7.2.1web_search网络搜索节点

- 获取state中用户的最新问题question
- 调用定义好的web_search_tool（使用TavilySearchResults进行搜索）对用户问题进行搜索
- 将搜索结果合并
- 将合并后的搜索结果转换为document格式
- 返回documents与question更新状态

```

def web_search(state):
    """
    基于优化后的问题进行网络搜索

    Args:
        state (dict): 当前图状态，包含优化后的问题

    Returns:
        state (dict): 更新后的状态，documents字段替换为网络搜索结果
    """
    log.info("---WEB SEARCH---") # 阶段标识
    question = state["question"] # 获取优化后的问题

    # 执行网络搜索
    docs = web_search_tool.invoke({"query": question}) # 调用网络搜索工具
    web_results = "\n".join([d["content"] for d in docs]) # 合并搜索结果
    web_results = Document(page_content=web_results) # 转换为文档格式

    return {"documents": web_results, "question": question} # 返回更新状态

```

7.2.2generate最终答案生成节点

- 获取state中的用户问题与document文档（可能是网络搜索得出，也可能是检索器检索得出）
- 定义提示词，告诉llm需要通过documents来回答用户question
- 将提示词、llm合并为处理链并解析llm的输出为str
- 调用处理链生成回答，将documents、question、generate返回更新state

```

def generate(state):
    """

```

```

生成回答
Args:
    state (dict): 当前图状态，包含问题和检索结果
Returns:
    state (dict): 更新后的状态，新增包含生成结果的generation字段
"""
question = state["question"] # 获取用户问题
documents = state["documents"] # 获取检索到的文档

prompt = PromptTemplate(
    template="你是一个问答任务助手。请根据以下检索到的上下文内容回答问题。如果不知道答案，请直接说明。回答保持简洁。\\n问题: {question} \\n上下文: {context} \\n回答: ",
    input_variables=["question", "context"],
)

# 后处理函数 - 格式化检索到的文档
def format_docs(docs):
    """将多个文档内容合并为一个字符串，用两个换行符分隔每个文档"""
    if isinstance(docs, list):
        return "\\n\\n".join(doc.page_content for doc in docs) # 拼接所有文档内容
    else:
        return "\\n\\n" + docs.page_content

# 构建RAG处理链
rag_chain = (
    prompt | # 第一步：使用提示模板
    llm | # 第二步：调用语言模型
    StrOutputParser() # 第三步：解析模型输出为字符串
)

# RAG生成过程
generation = rag_chain.invoke({"context": format_docs(documents), "question": question})
# 调用RAG链生成回答
return {"documents": documents, "question": question, "generation": generation} # 返回更新后的状态

```

7.2.3transform_query重新生成问题节点

- 获取用户问题、当前的documents、重新生成问题的次数（使用get可以设置默认为0）
- 定义提示词将llm定义为一个问题重写器
- 将提示词、llm合并为处理链并解析llm的输出为str
- 调用处理链生成新的问题，将documents、question（重新生成的）、重新生成的次数返回更新state

```

def transform_query(state):
    """
    优化用户问题，生成更合适的查询语句

    Args:
        state (dict): 当前图状态，包含用户问题和检索结果

    Returns:
        state (dict): 更新后的状态，question字段替换为优化后的问题
    """
    log.info("---TRANSFORM QUERY---") # 阶段标识
    question = state["question"] # 获取原始问题
    documents = state["documents"] # 获取当前文档
    transform_count = state.get("transform_count", 0)

    # 提示词模板 - 问题重写优化

```

```

system = """作为问题重写器，您需要将输入问题转换为更适合向量数据库检索的优化版本。\\n
    请分析输入问题并理解其背后的语义意图/真实含义。"""
re_write_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # 系统角色设定：问题优化器
        (
            "human", # 用户输入模板
            "这是初始问题：\\n\\n {question} \\n 请生成一个优化后的问题。",
        ),
    ]
)

# 构建问题重写处理链
question_rewriter = (
    re_write_prompt # 使用优化提示模板
    | llm # 调用语言模型
    | StrOutputParser() # 将输出解析为字符串
)

# 问题重写
better_question = question_rewriter.invoke({"question": question}) # 调用问题重写器
return {"documents": documents, "question": better_question, "transform_count":
transform_count+1} # 返回更新状态

```

7.2.4 retrieve 文档检索节点

- 获取用户问题
- 调用检索器工具，传入用户问题，获取相关文档
- 返回用户问题和检索到的documents，更新状态

```

def retrieve(state):
    """
    检索相关文档
    Args:
        state (dict): 当前图状态，包含用户问题

    Returns:
        state (dict): 更新后的状态，新增包含检索结果的documents字段
    """
    log.info("---去知识库中检索文档---") # 打印当前阶段标识
    question = state["question"] # 从状态中获取用户问题
    # 文档检索
    documents = retriever.invoke(question) # 调用检索器获取相关文档
    return {"documents": documents, "question": question} # 返回更新后的状态

```

7.2.5 garde_document 评价检索出的文档节点

1. 定义一个用于评分的llm
 - 定义数据模型类，用于规范llm的回答必须为所规定的内容，然后将llm与数据模型类绑定
 - 定义提示词，将llm定义为一个评估检索文档与用户问题相关性的评分器
 - 将提示词与llm绑定为处理链
2. 定义节点，筛选检索出的documents
 - 获取用户问题与待评估文档
 - 调用定义好的链，对每一个document进行评估，与用户问题相关则输出yes，否则输出no
 - 返回结果为yes时将document添加到列表中，no的不添加

- 返回过滤后的documents与用户问题，更新状态

```
# 数据模型 - 文档相关性评分
class GradeDocuments(BaseModel):
    """对检索到的文档进行相关性评分的二元判断"""

    binary_score: str = Field(
        description="文档是否与问题相关，取值为'yes'或'no'"
    )

# 带函数调用的LLM初始化
structured_llm_grader = llm.with_structured_output(GradeDocuments) # 绑定结构化输出到评分模型

# 提示词模板
system = """你是一个评估检索文档与用户问题相关性的评分器。
如果文档包含与用户问题相关的关键词或语义含义，则评为相关。
不需要非常严格的测试，目的是过滤掉错误的检索结果。
给出'yes'或'no'的二元评分来表示文档是否与问题相关。"""
grade_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # 系统角色提示
        ("human", "Retrieved document: \n\n {document} \n\n User question: {question}"), # 用户输入模板
    ]
)

# 构建检索评分器工作流
retrieval_grader_chain = grade_prompt | structured_llm_grader # 组合提示模板和LLM评分器

def grade_documents(state):
    """
    评估检索到的文档与问题的相关性

    Args:
        state (dict): 当前图状态，包含问题和检索结果

    Returns:
        state (dict): 更新后的状态，documents字段仅保留相关文档
    """
    log.info("---CHECK DOCUMENT RELEVANCE TO QUESTION---") # 打印当前阶段标识
    question = state["question"] # 获取用户问题
    documents = state["documents"] # 获取待评估文档

    # 文档评分与过滤
    filtered_docs = [] # 初始化相关文档列表
    for d in documents: # 遍历所有文档
        score = retrieval_grader_chain.invoke( # 调用评分器评估文档相关性
            {"question": question, "document": d.page_content}
        )
        grade = score.binary_score # 获取二元评分结果
        if grade == "yes": # 如果文档相关
            log.info("---GRADE: 打印相关标识---") # 打印相关标识
            filtered_docs.append(d) # 添加到相关文档列表
        else: # 如果文档不相关
            log.info("---GRADE: 打印不相关标识,并丢掉doc---") # 打印不相关标识
            continue # 跳过当前文档
    return {"documents": filtered_docs, "question": question} # 返回仅含相关文档的状态
```

7.3定义路由函数

由项目执行流程可知，图中一共需要三个路由函数：

1. 对话开始时由llm判断用户问题需要联网搜索还是使用检索器检索
2. 由generate节点生成回答后需要判断回答是否准确的回答了用户的问题
3. grade_document节点得到过滤后的documents后需要根据documents判断下一步流程

7.3.1对话开始时的路由函数

1. 定义一个用于决策的llm
 - 定义数据模型类，规定llm的输出必须为vectorstore或 web_search，然后将llm与数据模型类绑定
 - 定义提示词，告诉llm向量知识库中都由哪些类型的知识，当用户问到相关问题后llm会决策进入RAG，其他情况则用网络搜索
 - 将提示词与llm绑定为处理链
2. 定义路由函数
 - 获取用户问题
 - 调用处理链，传入question，得到llm回答
 - 回答为web_search则路由到web_search节点，回答为vectorstore则路由到retrieve节点

```
# 查询的动态路由： 根据用户的提问，决策采用哪种检索策略（网络检索，RAG）

# 数据模型
class RouteQuery(BaseModel):
    """将用户查询路由到最相关的数据源"""
    datasource: Literal["vectorstore", "web_search"] = Field(
        ...,
        description="根据用户问题选择将其路由到向量知识库或网络搜索",
    )

# 带函数调用的LLM
structured_llm_router = llm.with_structured_output(RouteQuery)

# 提示词模板
system = """你是一个擅长将用户问题路由到向量知识库或网络搜索的专家。
向量知识库包含与半导体材料，芯片制造，光刻技术相关的文档。
对于这些主题的问题请使用向量知识库，其他情况使用网络搜索。"""
route_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # 系统提示词
        ("human", "{question}"), # 用户问题占位符
    ]
)

# 创建问题路由链
question_router_chain = route_prompt | structured_llm_router

def route_question(state):
    """
    路由问题到网络搜索或RAG流程
    Args:
        state (dict): 当前图状态，包含用户问题

    Returns:
        str: 下一节点的名称（web_search或vectorstore）
    """
```

```

"""
log.info("---ROUTE QUESTION---") # 阶段标识
question = state["question"] # 获取用户问题
source = question_router_chain.invoke({"question": question}) # 调用问题路由器

# 根据路由结果决定下一个节点
if source.datasource == "web_search":
    log.info("---路由到web搜索---")
    return "web_search"
elif source.datasource == "vectorstore":
    log.info("---路由到RAG系统---")
    return "vectorstore"

```

7.3.2生成回答后的路由函数

1. 定义一个判断llm是否产生幻觉的llm

- 定义数据模型类，规定llm的输出必须为yse或 no，然后将llm与数据模型类绑定
- 定义提示词，将llm定义为一个评估回答是否基于事实（documents），若返回no，则llm产生了幻觉
- 将提示词与llm绑定为处理链

```

# 数据模型 - 生成内容幻觉评分
class GradeHallucinations(BaseModel):
    """对生成回答中是否存在幻觉进行二元评分"""

    binary_score: str = Field(
        description="回答是否基于事实，取值为'yes'或'no'"
    )

# 带函数调用的LLM初始化
structured_llm_grader = llm.with_structured_output(GradeHallucinations) # 绑定结构化输出到评分模型

# 提示词模板
system = """您是一个评估生成内容是否基于检索事实的评分器。\\n
    给出'yes'或'no'的二元评分。'yes'表示回答是基于/支持于给定事实集的。"""
hallucination_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # 系统角色设定
        ("human", "事实集: \\n\\n {documents} \\n\\n 生成内容: {generation}"), # 用户输入模板
    ]
)

# 构建幻觉检测 workflow
hallucination_grader_chain = (
    hallucination_prompt # 使用幻觉检测提示模板
    | structured_llm_grader # 调用结构化评分的LLM
)

```

2. 定义一个用于判断最终回答是否解决用户问题的llm

- 定义数据模型类，规定llm的输出必须为yse或 no，然后将llm与数据模型类绑定
- 定义提示词，将llm定义为一个评估回答是否解决用户问题的评分器
- 将提示词与llm绑定为处理链

```

# 数据模型 - 回答质量评分
class GradeAnswer(BaseModel):
    """评估回答是否解决用户问题的二元评分模型"""

    binary_score: str = Field(

```

```

        description="回答是否解决了问题，取值为'yes'或'no'"
    )

# 初始化带函数调用的LLM
structured_llm_grader = llm.with_structured_output(GradeAnswer) # 绑定结构化输出到评分模型

# 提示词模板
system = """您是一个评估回答是否解决用户问题的评分器。\\n
    给出'yes'或'no'的评分。'yes'表示:回答确实解决了该问题。"""
answer_prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system), # 系统角色设定
        ("human", "用户问题: \\n\\n {question} \\n\\n 生成回答: {generation}"), # 用户输入模板
    ]
)

# 构建回答质量评估工作流
answer_grader_chain = (
    answer_prompt # 使用回答评估提示模板
    | structured_llm_grader # 调用结构化评分的LLM
)

```

3. 定义路由函数

- 获取用户问题、参考文档documents（可能来自网络搜索，有可能来自RAG）、生成的结果
- 调用评估回答是否基于事实（documents）的处理链，传入documents和generation
- 返回yes，则说明回答没有产生幻觉，则调用评估最终回答是否解决用户问题的处理链
 - 返回yes，则说明生成内容准确回答问题，路由到END节点
 - 返回no，则说明生成内容未能准确回答问题，返回no useful，路由到transform_query节点重新生成问题后，进入向量数据库中检索
- 返回no，则说明生成内容未基于参考文档，返回not supported，将路由到generate节点重新尝试

```

def grade_generation_v_documents_and_question(state):
    """
    评估生成结果是否基于文档并正确回答问题
    Args:
        state (dict): 当前图状态，包含问题、文档和生成结果
    Returns:
        str: 下一节点的名称（useful/not useful/not supported）
    """
    log.info("---检查生成内容是否存在幻觉---") # 阶段标识
    question = state["question"] # 获取用户问题
    documents = state["documents"] # 获取参考文档
    generation = state["generation"] # 获取生成结果

    # 检查生成是否基于文档
    score = hallucination_grader_chain.invoke({"documents": documents, "generation":
generation})
    grade = score.binary_score

    if grade == "yes": # 如果生成基于文档
        log.info("---判定: 生成内容基于参考文档---")
        # 检查是否准确回答问题
        log.info("---评估: 生成回答与问题的匹配度---")
        score = answer_grader_chain.invoke({"question": question, "generation": generation})
        grade = score.binary_score
        if grade == "yes": # 如果正确回答问题
            log.info("---判定: 生成内容准确回答问题---")
            return "useful" # 返回有用结果

```

```

else: # 如果没有回答问题
    log.info("---判定：生成内容未能准确回答问题---")
    return "not useful" # 返回无用结果
else: # 如果生成不基于文档
    log.info("---判定：生成内容未基于参考文档，将重新尝试---")
    return "not supported" # 返回不支持结果

```

7.3.3 过滤检索器得到的documents后的路由函数

- 获取过滤后的文档filtered_documents，重新生成问题的次数transform_count
- 如果filtered_documents为空，则说明RAG没有与问题相关的文档
 - 此时如果transform_count >= 2，说明已经检索过2次，则路由到web_search节点，转为web查询问题
 - 检索次数还不到2次，则路由到transform_query节点，优化问题
- 如果filtered_documents不为空，则路由到generate节点，生成最终回答

```

def decide_to_generate(state):
    """
    决定是生成回答还是重新优化问题

    Args:
        state (dict): 当前图状态，包含问题和过滤后的文档

    Returns:
        str: 下一节点的名称（transform_query或generate）
    """
    log.info("---ASSESS GRADED DOCUMENTS---") # 阶段标识

    filtered_documents = state["documents"] # 获取已过滤文档
    transform_count = state.get("transform_count", 0)

    if not filtered_documents: # 如果没有相关文档
        if transform_count >= 2:
            log.info("---决策：所有文档都与问题无关，并且已经循环了2次，转为web查询问题---")
            return "web_search" # 返回问题优化节点
            log.info("---决策：所有文档都与问题无关，将转换查询问题---")
            return "transform_query" # 返回问题优化节点
        else: # 如果有相关文档
            log.info("---决策：生成最终回答---")
            return "generate" # 返回回答生成节点

```

7.4 绘制图

1. 初始化 workflow
2. 添加各状态节点：网络搜索节点、查询优化节点、文档检索节点、文档相关性评分节点、回答生成节点
3. 添加固定边：web_search->generate、transform_query->retrieve、retrieve->grade_documents
4. 添加条件边：START->web_search/retrieve、grade_documents->web_search/transform_query/generate、generate->generate/transform_query/END
5. 编译图
6. 执行图

```

# 初始化 workflow
workflow = StateGraph(GraphState)

# 定义各状态节点
workflow.add_node("web_search", web_search) # 网络搜索节点
workflow.add_node("retrieve", retrieve) # 文档检索节点

```

```

workflow.add_node("grade_documents", grade_documents) # 文档相关性评分节点
workflow.add_node("generate", generate) # 回答生成节点
workflow.add_node("transform_query", transform_query) # 查询优化节点

# 起始路由判断
workflow.add_conditional_edges(
    START,
    route_question,
    {
        "web_search": "web_search",
        "vectorstore": "retrieve"
    }
)

# 添加固定边
workflow.add_edge("web_search", "generate") # 网络搜索后直接生成回答
workflow.add_edge("retrieve", "grade_documents") # 检索后评估文档相关性

# 文档评估后的条件分支
workflow.add_conditional_edges(
    'grade_documents',
    decide_to_generate
)

# 生成结果评估后的条件分支
workflow.add_conditional_edges(
    "generate", # 生成节点
    grade_generation_v_documents_and_question, # 生成质量评估函数
    {
        "not supported": "generate", # 生成不符合要求时重试
        "useful": END, # 生成符合要求时结束
        "not useful": "transform_query", # 生成无用结果时优化查询
    },
)

workflow.add_edge("transform_query", "retrieve") # 查询优化后重新检索

# 编译工作流
graph = workflow.compile()

# draw_graph(graph, 'graph_rag2.png')

# 执行工作流

_printed = set() # set集合，避免重复打印

# 执行工作流
while True:
    question = input('用户: ')
    if question.lower() in ['q', 'exit', 'quit']:
        print('对话结束，拜拜! ')
        break
    else:
        inputs = {
            "question": question
        }
        # 流式执行工作流
        for output in graph.stream(inputs):
            for key, value in output.items():
                # 打印当前节点名称
                pprint(f"Node '{key}':") # 显示当前执行的节点名称
                # 可选：打印每个节点的完整状态信息

```

```
        # pprint.pprint(value["keys"], indent=2, width=80, depth=None)
    pprint("\n---\n") # 节点分隔线

# 打印最终生成结果
pprint(value["generation"]) # 输出最终生成的回答内容
```